

DemiEngine Technical Documentation

DemiEngine Project

Current engine snapshot

Contents

1	Overview	1
1.1	Current Goals	1
1.2	Major Features	1
1.3	Current Limitations	2
2	Build and Command Line	3
2.1	Build Requirements	3
2.2	Standard Commands	3
2.3	Build Targets	3
3	Architecture	5
3.1	High Level Structure	5
3.2	Runtime Loop	5
3.3	World Model	5
3.4	Design Philosophy	6
4	Data Formats	7
4.1	Project Files	7
4.2	Scene Files	7
4.3	HUD Files	7
4.4	Save Files	8
4.5	Asset Manifests	8
5	Runtime Systems	9
5.1	Rendering	9
5.2	Physics	9
5.3	Audio	9
5.4	Window and Runtime Control	10
5.5	Scene Switching	10
6	Lua API	11
6.1	Lifecycle	11
6.2	Services	11
6.3	Save API	12
6.4	HUD API	12
6.5	Scene Switching	12
7	HUD and Menu System	13
7.1	Two HUD Workflows	13
7.2	Declarative HUD Elements	13
7.3	Canvas Size	13
7.4	Groups	13

7.5	Button Events	14
7.6	Event Annotations	14
8	Saves and Settings	15
8.1	Save File Shape	15
8.2	Settings Persistence	15
8.3	Current Save API Scope	15
9	Minimal 2D Example	16
9.1	Project Contents	16
9.2	Levels and Scene Switching	16
9.3	Gameplay	17
9.4	Menu	17
10	Testing and Validation	18
10.1	Validation	18
10.2	Automated Tests	18
10.3	Recommended Checks After Data Changes	18
A	File Reference	19
A.1	Core Source Files	19
A.2	Example Files	19

Chapter 1

Overview

DemiEngine is a small C++20 and Lua game engine scaffold focused on deterministic, AI-friendly development. The current repository implements a playable 2D example, a runtime window, Lua gameplay scripting, data validation, asset manifests, simple physics, audio playback, save files, and a declarative HUD system.

The engine is intentionally compact. Most systems are implemented as readable runtime services rather than large frameworks. This makes the codebase suitable for iterative engine development and for agents or humans to inspect, modify, and validate changes through the command line.

1.1 Current Goals

The current implementation emphasizes:

- stable JSON source data for projects, scenes, HUD layouts, assets, and saves;
- Lua scripts for game-specific behavior;
- deterministic example data and small diffs;
- a command line validation path shared by runtime and development workflows;
- a minimal 2D runtime that can be extended into a broader game engine.

1.2 Major Features

The implemented feature set includes:

- project files using `*.project.json`;
- scene files using `*.scene.json`;
- HUD files using `*.hud.json`;
- save files using `*.save.json`;
- asset manifests using `*.asset.json`;
- SDL3 rendering and window management when SDL3 is available;
- Lua 5.4 script hosting when Lua is available;
- MP3 playback through libmpg123 when available;
- AABB based 2D physics with gravity, static colliders, dynamic rigidbodies, and bounciness;
- a built-in pixel text renderer for HUD text;
- declarative HUD rectangles, text, and buttons;
- Lua UI events for HUD buttons;
- Lua save access for persistent settings;
- an example game at `examples/minimal\_2d`.

1.3 Current Limitations

The engine is still early. Important current limitations are:

- JSON parsing is intentionally lightweight and not a full general JSON parser;
- runtime scene switching is not fully implemented;
- rendering is 2D oriented with simple fallback shapes and PPM texture loading;
- the editor executable is currently only a scaffold;
- UI layout supports fixed virtual canvas coordinates, not a full anchor/layout system;
- Lua bindings are explicit and small, not a complete engine API surface;
- asset handling supports stable IDs and simple manifests, but not a full import pipeline.

Chapter 2

Build and Command Line

2.1 Build Requirements

The project uses CMake and C++20. Optional runtime functionality is enabled when dependencies are available:

- SDL3 enables the runtime window and renderer;
- Lua 5.4 enables game scripts;
- libmpg123 enables MP3 audio playback.

Missing optional dependencies do not prevent the project from compiling. Instead, the corresponding runtime feature is compiled out or skipped.

2.2 Standard Commands

The expected development commands are:

```
cmake --preset linux-debug
cmake --build --preset linux-debug
ctest --preset linux-debug
./build/linux-debug/demi --help
./build/linux-debug/demi validate examples/minimal_2d/demi.project.json
./build/linux-debug/demi script check examples/minimal_2d/scripts/player.lua
./build/linux-debug/demi lua-stubs generate
```

The runtime can be launched with:

```
./build/linux-debug/demi run --project examples/minimal_2d/demi.project.json
```

For headless smoke testing, SDL can use its dummy video driver:

```
SDL_VIDEODRIVER=dummy ./build/linux-debug/demi run --project examples/minimal_2d/demi.project.json --frames 5
```

2.3 Build Targets

The current CMake configuration defines these major targets:

demi-core	Core validation, filesystem, diagnostics, and asset registry code.
demi-runtime-lib	Runtime systems including scene loading, rendering, physics, audio, and Lua hosting.

<code>demi</code>	Main command line executable.
<code>demi-runtime</code>	Runtime-specific executable entry point.
<code>demi-editor</code>	Editor scaffold.
<code>demi-smoke-tests</code>	Core smoke tests.
<code>demi-physics2d-tests</code>	Physics regression tests.

Chapter 3

Architecture

3.1 High Level Structure

The engine is split into a small set of modules under `src/demi`:

- **assets**: asset manifest loading and reference extraction;
- **diagnostics**: structured diagnostics and severity reporting;
- **filesystem**: known source file discovery and file kind helpers;
- **schema**: validation of project, scene, HUD, save, and asset source data;
- **runtime**: scene loading, world data, rendering, physics, audio, Lua, and application loop;
- **cli**: command line entry point;
- **editor**: editor scaffold.

3.2 Runtime Loop

The runtime performs the following steps:

1. load and validate the project file;
2. load the main scene and optional HUD file;
3. create the SDL window and renderer if SDL3 is available;
4. load texture and audio assets from asset manifests;
5. initialize Lua and load project, entity, and HUD button scripts;
6. run fixed updates and physics at a fixed step;
7. run Lua update callbacks;
8. apply requested runtime changes such as window mode;
9. render the world and HUD;
10. shut down scripts, audio, renderer, and window.

3.3 World Model

The runtime world is represented by `World` in `src/demi/runtime/SceneData.h`. It contains:

- scene metadata;
- the HUD canvas size;
- entities and optional components;
- HUD rectangles, buttons, and text;
- debug lines.

Entities are simple value objects with optional components. The current component set includes transforms, cameras, sprites, Lua scripts, rigidbodies, box colliders, isometric grid data, hitbox controller data, and buildable metadata.

3.4 Design Philosophy

The current implementation prefers explicit, small APIs. New engine capabilities are expected to follow this pattern:

- add source data fields first where possible;
- validate data through the CLI;
- expose only deliberate Lua APIs;
- keep generated outputs out of source directories;
- keep runtime/editor behavior backed by the same data and diagnostics paths.

Chapter 4

Data Formats

All source data files are JSON-like engine data and must include `format\_version`. The validator recognizes project, scene, HUD, save, and asset manifest files.

4.1 Project Files

Project files use `*.project.json`. A project declares a name, main scene, scene list, and optional scripting entry point.

```
{
  "format_version": 1,
  "name": "Minimal 2D",
  "main_scene": "scene://minimal_2d/main",
  "scenes": [
    { "id": "scene://minimal_2d/main", "path": "scenes/main.scene.json" }
  ],
  "scripting": {
    "language": "lua54",
    "entry": "script://scripts/game.lua"
  }
}
```

4.2 Scene Files

Scene files use `*.scene.json`. They declare a scene ID, scene name, optional HUD file, and an entity list.

```
{
  "format_version": 1,
  "id": "scene://minimal_2d/main",
  "name": "Minimal Main",
  "hud": "main.hud.json",
  "entities": []
}
```

4.3 HUD Files

HUD files use `*.hud.json`. They declare a virtual canvas and HUD elements.

```
{
  "format_version": 1,
```

```

"canvas_size": [960.0, 540.0],
"elements": [
  {
    "type": "button",
    "id": "menu_button_start",
    "group": "menu_main",
    "label": "START GAME",
    "position": [270.0, 226.0],
    "size": [420.0, 52.0],
    "action": "menu_button_start"
  }
]
}

```

The HUD canvas size is per HUD file. A game may author one HUD at 960 by 540 and another at 1920 by 1080. The renderer scales from the HUD canvas to the actual window size, and HUD mouse hit testing maps back into the same canvas.

Button actions are handled by Lua functions annotated with `@HandleAction("action");` the annotation value should match the HUD element's `action` field. General events can use `@OnEvent("event");` modules that contain annotations should be listed in the project file's `scripting.modules` array.

4.4 Save Files

Save files use `*.save.json`. They contain a slot name and arbitrary state fields.

```

{
  "format_version": 1,
  "slot": "settings",
  "state": {
    "master_volume": 0.8,
    "window_mode": "fullscreen"
  }
}

```

Lua can read and write simple string and number values in these files through the `Save` service.

4.5 Asset Manifests

Asset manifests use `*.asset.json`. They define stable `asset://` IDs and source paths. The current runtime loads PPM texture data for sprites and MP3 data for audio if `libmpg123` is available.

Chapter 5

Runtime Systems

5.1 Rendering

The renderer is implemented in `src/demi/runtime/Renderer2D.cpp`. It uses SDL3 when available. Current renderer capabilities include:

- clearing with the active camera color;
- drawing PPM-backed textures when available;
- drawing fallback rectangles for entities without textures;
- drawing collider outlines for debug visibility;
- drawing debug lines;
- drawing HUD rectangles, buttons, and pixel text;
- scaling HUD coordinates from each HUD file canvas to the active window size.

The HUD text renderer is a built-in pixel font. It currently supports the glyphs needed by the example menu and HUD.

5.2 Physics

The 2D physics system is implemented in `src/demi/runtime/Physics2D.cpp`. It supports:

- dynamic and static rigidbodies;
- gravity scale per dynamic body;
- axis-aligned box colliders;
- static collider resolution;
- bounciness on vertical contacts;
- overlap box queries exposed to Lua;
- regression coverage for edge-contact teleport behavior.

The resolver uses previous collider bounds for side decisions, which avoids incorrectly snapping a player across a platform when touching a top-right or top-left edge.

5.3 Audio

The audio system is implemented in `src/demi/runtime/AudioSystem.cpp`. It supports:

- asset-backed audio loading;
- MP3 decoding with libmpg123 when available;
- playing and stopping audio handles from Lua;
- master volume control;
- mixing currently playing sounds into the SDL audio callback.

5.4 Window and Runtime Control

The runtime supports Lua-requested window mode changes:

- `windowed`;
- `borderless`;
- `fullscreen`.

Window mode changes are requested from Lua and applied by the runtime loop. Lua can also pause or resume physics through `Runtime.set\_physics\_enabled` and request shutdown through `Runtime.quit`.

5.5 Scene Switching

Runtime scene switching is implemented in `src/demi/runtime/SceneLoader.cpp` and `src/demi/runtime/LuaS`. The free function `loadScene` resolves a scene id against the project scene list, validates the scene and optional HUD, and returns a parsed `World`.

Lua calls `Scene.load` from any callback. The host records the request as pending. The application loop in `src/demi/runtime/RuntimeApp.cpp` checks for a pending scene after the Lua update step and, if present, rebuilds the active world, destroys and re-loads scripts, and re-runs `on_create/on_start`. Switches are applied between frames so they never re-enter Lua mid-callback.

Chapter 6

Lua API

Lua 5.4 scripts are loaded when Lua is available. Scripts return a table and may implement lifecycle functions.

Scripts attached to the project, entities, and HUD buttons are checked for direct file times-tamp changes each update. When a script file changes, the runtime calls `on\_destroy` on the old table, loads the new table, restores the engine-provided fields, and calls `on\_create` and `on\_start`. Lua syntax can be checked from automation with `demi script check <script>`. LuaLS/EmmyLua annotations for the exposed runtime API are maintained in `scripts/stubs/demi.lua`; `demi lua-stubs generate [path]` copies that checked-in stub file to a requested location.

6.1 Lifecycle

Supported lifecycle functions are:

```
function Script:on_create() end
function Script:on_start() end
function Script:on_update(dt) end
function Script:on_fixed_update(dt) end
function Script:on_destroy() end
```

HUD button scripts may additionally implement:

```
function Script:on_ui_hover(event) end
function Script:on_ui_click(event) end
```

The UI event contains `id`, `label`, `mouse\_x`, and `mouse\_y`. Mouse coordinates are in the HUD file canvas coordinate system.

6.2 Services

The current Lua services include:

Debug	Logging, debug lines, and clearing debug lines.
Input	Keyboard, mouse buttons, mouse position, mouse world position, viewport size.
Entity	Entity lookup, creation, destruction, and compatibility transform position access.
Transform	Position, rotation, and scale access for entities with <code>Transform2D</code> .
Timer	Delayed and repeating callbacks cancellable by timer id.

Events	In-process named event subscriptions and payload delivery.
Rigidbody2D	Get and set velocity and apply impulses.
Physics2D	Overlap box queries.
Hud	Imperative HUD text, rects, visibility, colors, groups, and text updates.
Save	Simple number and string persistence in save slots.
Audio	Play, stop, master volume get and set.
Runtime	Quit, physics enable, window mode get and set.
Scene	Runtime scene switching by scene id.

6.3 Save API

The save API is designed for small game settings and simple persistent values.

```
local volume = Save.get_number("settings", "master_volume", 1.0)
Save.set_number("settings", "master_volume", volume)

local mode = Save.get_string("settings", "window_mode", "windowed")
Save.set_string("settings", "window_mode", mode)
```

6.4 HUD API

HUD elements can be generated or updated from Lua:

```
Hud.text("points", "POINTS: 0", 24.0, 24.0, 3.0)
Hud.rect("panel", 20.0, 20.0, 320.0, 120.0, 0.1, 0.1, 0.2, 0.8)
Hud.set_text("points", "POINTS: 10")
Hud.set_rect("bar", 100.0, 100.0, 200.0, 16.0)
Hud.set_color("menu_tab_sound", 0.24, 0.27, 0.46, 0.94)
Hud.set_visible("points", true)
Hud.set_group_visible("menu_main", false)
```

6.5 Scene Switching

`Scene.load` requests a runtime scene switch by scene id. The switch is deferred: it is applied between frames by the runtime loop, so it is safe to call from any Lua callback (menu buttons, update, fixed update).

```
Scene.load("scene://minimal_2d/spiral")
```

When a scene switch is applied, the runtime:

1. validates and parses the target scene (and its optional HUD) by id from the project scene list;
2. calls `on_destroy` on every active script and releases their script tables;
3. replaces the active world in place (so existing world references stay valid);
4. re-loads the project entry script, entity scripts, and HUD button scripts;
5. calls `on_create` and `on_start` on the freshly loaded scripts.

Required modules stay cached across a switch, so a shared state module can carry a pending flag (for example an `auto\_start` flag) from the button that requested the switch into the new scene's `on_start`. Runtime-created entities from the previous scene are discarded with the old world.

Chapter 7

HUD and Menu System

7.1 Two HUD Workflows

DemiEngine currently supports two HUD workflows:

- imperative HUD creation through Lua functions such as `Hud.text` and `Hud.rect`;
- declarative HUD layout through `*.hud.json` files.

The declarative path is preferred for authored menus. The imperative path remains useful for generated UI, temporary debug HUDs, and dynamic game-specific overlays.

7.2 Declarative HUD Elements

HUD files currently support three element types:

<code>rect</code>	A colored rectangle with position, size, color, visibility, and group.
<code>text</code>	Pixel font text with position, scale, color, visibility, and group.
<code>button</code>	A clickable rectangle with label, colors, script callback, visibility, and group.

7.3 Canvas Size

Each HUD file can define its own coordinate system:

```
"canvas_size": [1920.0, 1080.0]
```

All positions and sizes in that HUD file are authored against this canvas. At runtime, the renderer scales the HUD to the actual window size. HUD click detection uses the same conversion, so fullscreen and resized windows remain interactive.

7.4 Groups

Groups are inspired by Unity-style parent UI objects. Instead of keeping Lua arrays of every element ID, elements declare a `group`. Lua then toggles a whole group:

```
Hud.set_group_visible("menu_main", true)
Hud.set_group_visible("menu_options", false)
```

The `minimal\_2d` menu uses these groups:

- `menu\_base`;
- `menu\_main`;

- menu_options;
- menu_sound;
- menu_video;
- menu_dropdown;
- game_hud.

7.5 Button Events

Buttons can declare an action annotation:

```
{
  "type": "button",
  "id": "menu_button_start",
  "group": "menu_main",
  "label": "START GAME",
  "action": "menu_button_start"
}
```

The runtime dispatches the action to Lua functions annotated with `@HandleAction`:

```
-- @HandleAction("menu_button_start")
function Menu.start_game(event)
  Scene.load("scene://game/level_1")
end
```

The lower-level hud_action event is also emitted for systems that prefer event-bus routing.

7.6 Event Annotations

Lua functions can subscribe to game events with `@OnEvent` instead of calling `Events.subscribe` manually:

```
-- @OnEvent("player_died")
function GameEvents.show_game_over(event)
  Hud.set_text("game_over_points", "POINTS: " .. tostring(event.points))
end
```

For reusable modules loaded with `require`, list the module in `scripting.modules` in the project file so the runtime can register its annotations.

Chapter 8

Saves and Settings

8.1 Save File Shape

Save files are source-like JSON files with a stable format:

```
{
  "format_version": 1,
  "slot": "settings",
  "state": {
    "master_volume": 1.0,
    "window_mode": "windowed"
  }
}
```

The Lua save service currently supports flat number and string values in the `state` object.

8.2 Settings Persistence

The `minimal\_2d` example uses save slot `settings`. On menu startup it loads:

- `master\_volume`;
- `window\_mode`.

When the user changes volume or window mode in the options menu, Lua immediately updates the save file. The runtime applies the loaded values on startup.

8.3 Current Save API Scope

The save API is intentionally small. It is suitable for settings and simple values, but it is not yet a full save-game serialization system. Future work should decide whether structured tables, arrays, entity state snapshots, and schema-aware save migrations belong in this API or in a higher-level gameplay save service.

Chapter 9

Minimal 2D Example

The `examples/minimal\_2d` project is the main gameplay example. It demonstrates project loading, scene data, generated gameplay objects, Lua scripts, physics, audio, saves, and the HUD menu system.

9.1 Project Contents

Important files include:

<code>demi.project.json</code>	Project entry point.
<code>scenes/main.scene.json</code>	Level 1 scene and entity definitions.
<code>scenes/spiral.scene.json</code>	Level 2 scene: a procedurally generated upward spiral climb.
<code>scenes/main.hud.json</code>	Declarative main menu, level select, options UI, and points HUD.
<code>scripts/game.lua</code>	Shared game controller: points, coins, platform tracking, and level dispatch.
<code>scripts/level_main.lua</code>	Level 1 generation and camera (horizontal platforms).
<code>scripts/level_spiral.lua</code>	Level 2 generation and camera (upward spiral climb).
<code>scripts/game_state.lua</code>	Shared state across scene switches (level, <code>auto_start</code> , fall floor).
<code>scripts/player.lua</code>	Player update entry point.
<code>scripts/player_platformer.lua</code>	Platformer movement and respawn helpers.
<code>scripts/player_slingshot.lua</code>	Slingshot aim, recharge, launch, and trajectory preview.
<code>scripts/main_menu.lua</code>	Menu state controller, level select, and settings persistence.
<code>scripts/menu_button.lua</code>	HUD button event dispatcher.
<code>saves/settings.save.json</code>	Generated or updated settings save file.

9.2 Levels and Scene Switching

The example ships two levels. The main menu has a Level Select screen showing one card per level (number, name, description, a mini preview, and a PLAY button). Selecting a level calls `Scene.load` with the target scene id; the runtime rebuilds the world and scripts between frames (see the Scene Switching section in the Lua API chapter). A shared state module carries an `auto\_start` flag from the menu button into the new scene's `on\_start` so the selected level

begins immediately. On the Level Select screen, keyboard 1/2 pick a level and **Escape** returns to the main menu.

- **Level 1 – Orbit Run** (`scenes/main.scene.json`): the original deterministic horizontal slingshot platformer. Platforms generate ahead of the camera along the x axis.
- **Level 2 – Spiral Climb** (`scenes/spiral.scene.json`): a procedurally generated upward spiral. Platforms wind around a center point at a fixed angle step with a slowly growing radius, ascending by a fixed height per step. The camera follows the player on both axes. The fall floor is the same fixed value as Level 1, so a missed jump respawns at the start just like the main level.

9.3 Gameplay

The example is a 2D slingshot platformer. The player can move, jump, bounce on platforms, collect coins, and use a slingshot mechanic. Platform generation is deterministic and happens ahead of the camera.

Recent gameplay behavior includes:

- slingshot can recharge after touching or bouncing off platforms;
- mid-air slingshot preserves bounce velocity;
- trajectory preview uses snapshotted velocity for a stable prediction line;
- platform edge collision avoids teleporting across the platform.

9.4 Menu

The main menu is authored in `main.hud.json`. Lua controls group visibility and dynamic settings. The menu includes:

- cosmic themed start screen;
- Start Game, Options, and Quit buttons;
- Sound and Video tabs;
- master volume controls;
- window mode dropdown for windowed, borderless, and fullscreen;
- persistent settings through the **Save** service.

Chapter 10

Testing and Validation

10.1 Validation

The validator checks known source data files. It currently recognizes:

- *.project.json;
- *.scene.json;
- *.hud.json;
- *.save.json;
- *.asset.json.

The validator enforces `format\_version` and required fields for each file kind. For scenes and HUD files, it also validates referenced Lua scripts and asset IDs where those references are present in the text.

10.2 Automated Tests

Current tests include:

- `demi-smoke-tests`;
- `demi-physics2d-tests`;
- `demi-runtime-minimal-frame`.

The runtime frame test uses SDL's dummy video driver and launches `minimal\_2d` for one frame.

10.3 Recommended Checks After Data Changes

After editing example project data, run:

```
./build/linux-debug/demi validate examples/minimal_2d
```

After editing runtime code, run:

```
cmake --build --preset linux-debug  
ctest --preset linux-debug
```

Appendix A

File Reference

A.1 Core Source Files

<code>src/demi/assets/AssetRegistry.*</code>	Asset manifests and reference extraction.
<code>src/demi/diagnostics/Diagnostic.*</code>	Diagnostic data and helpers.
<code>src/demi/filesystem/ProjectPaths.*</code>	Known source file classification and collection.
<code>src/demi/schema/Validation.*</code>	Source data validation.
<code>src/demi/runtime/SceneData.h</code>	Runtime data model and components.
<code>src/demi/runtime/SceneLoader.cpp</code>	Project, scene, and HUD loading.
<code>src/demi/runtime/RuntimeApp.cpp</code>	Main runtime loop.
<code>src/demi/runtime/Renderer2D.*</code>	SDL renderer and HUD drawing.
<code>src/demi/runtime/Physics2D.*</code>	Physics stepping and queries.
<code>src/demi/runtime/AudioSystem.*</code>	Audio asset loading and playback.
<code>src/demi/runtime/LuaScriptHost.*</code>	Lua services, lifecycle, UI events, and saves.

A.2 Example Files

<code>examples/minimal_2d/demi.project.json</code>	Example project file.
<code>examples/minimal_2d/scenes/main.scene.json</code>	Main scene.
<code>examples/minimal_2d/scenes/main.hud.json</code>	Main HUD and menu layout.
<code>examples/minimal_2d/scripts/game.lua</code>	Main game script.
<code>examples/minimal_2d/scripts/main_menu.lua</code>	Menu state and settings controller.
<code>examples/minimal_2d/scripts/menu_button.lua</code>	HUD button click dispatcher.
<code>examples/minimal_2d/scripts/player.lua</code>	Player script.
<code>examples/minimal_2d/scripts/player_platformer.lua</code>	Movement and platform contact helpers.
<code>examples/minimal_2d/scripts/player_slingshot.lua</code>	Slingshot behavior.
<code>examples/minimal_2d/saves/settings.save.json</code>	User settings save slot.